

NRF24L01+ Time Synchronization for Bimanual Grippers

RF-Based Pi-to-Pi Clock Sync with Enhanced ShockBurst

Contents

1	Introduction	2
2	NRF24L01+ Hardware Overview	2
2.1	Module Variants	2
2.2	Pin Mapping	3
2.3	Wiring Schematic	3
2.4	SPI Configuration	3
2.5	Updated Single-Gripper Hardware Architecture	4
2.6	Updated Bill of Materials	4
3	Enhanced ShockBurst Protocol	4
3.1	On-Air Packet Format	5
3.2	Packet Control Field (PCF) Breakdown	5
3.3	Auto-ACK and Retransmission	5
3.4	ESB Timing	5
3.5	Data Rate Selection	5
3.6	ACK Payload Feature	6
4	Ping-Pong Protocol Design	6
4.1	Approach 1: Software Ping-Pong	6
4.2	Approach 2: Hardware ACK Payload	7
4.3	Approach Comparison	7
4.4	Pipe Configuration	7
4.5	NRF24L01+ Packet Format	8
5	Time Synchronization Algorithm	8
5.1	NTP-Style Four-Timestamp Principle	8
5.2	Mathematical Derivation	8
5.3	Multi-Round Averaging with Outlier Rejection	9
5.4	Synchronization Algorithm	9
5.5	Worked Numerical Example	9
6	Dual-Camera Integration	11
6.1	Camera Overview	11
6.2	Camera Specifications	11
6.3	Updated Software Architecture	11
6.4	Frame Synchronization Strategy	11
6.5	Updated LeRobot Dataset Columns	12

7	Error Handling and Drift Correction	12
7.1	Packet Loss Scenarios	12
7.2	Protocol State Machine	12
7.3	Retry and Timeout Configuration	12
7.4	Clock Drift Correction	12
7.5	Packet Loss Recovery	14
8	Complete Peer Implementation	14
8.1	Radio Initialization (Shared)	14
8.2	Node A — Initiator Main Loop	14
8.3	Node B — Responder Main Loop	14
8.4	Packet Encoding	14
9	Performance Analysis	17
9.1	Latency Breakdown	17
9.2	Synchronization Error Comparison	17
9.3	Step Timing at 30 Hz	17
9.4	Power Consumption	17
10	References and Hardware Links	18
10.1	QR Code Quick Links	18
10.2	Reference URLs	18

1 Introduction

Bimanual data collection for imitation learning requires two grippers to capture synchronized frames at 30 Hz. Any timing drift between the two Raspberry Pi units causes frame misalignment that degrades policy quality. Our previous system used WiFi Direct with UDP triggers, achieving ~ 2 ms bounded timing error. This document presents an alternative synchronization approach using **NRF24L01+ wireless modules**, which achieves **~ 0.8 ms** timing error with significantly lower power consumption and simpler network setup.

The NRF24L01+ is a 2.4 GHz ISM-band transceiver with a hardware-accelerated protocol layer called **Enhanced ShockBurst** (ESB). ESB provides automatic CRC, acknowledgement, and retransmission—features that would require significant software overhead with raw WiFi sockets. The module communicates with the RPi over SPI at up to 10 MHz, and its 32-byte maximum payload size is sufficient for our 28-byte synchronization packet with 4 bytes of headroom.

Table 1: WiFi UDP vs NRF24L01+ comparison for peer-to-peer synchronization.

Property	WiFi UDP	NRF24L01+
Round-trip latency	~ 2 ms	~ 0.8 ms
Setup complexity	WiFi Direct config	SPI enable only
Power (active)	~ 300 mA	~ 12 mA
Power (idle)	~ 100 mA	$\sim 26 \mu\text{A}$
Range (indoor)	~ 30 m	~ 10 m (basic) / ~ 100 m (PA+LNA)
Payload size	~ 1500 B (MTU)	32 B max
Frequency band	2.4 GHz WiFi	2.4 GHz ISM (non-WiFi)
Protocol overhead	IP + UDP headers	ESB (hardware)

Design Motivation

Our 28-byte sync packet fits comfortably within the NRF24L01+ 32-byte maximum payload. The hardware ACK+payload feature allows the responder to piggyback data on the automatic acknowledgement, eliminating the need for explicit mode switching. This yields round-trip communication in a single radio transaction ($\sim 557 \mu\text{s}$ theoretical, ~ 0.8 ms measured).

Scope. This document covers NRF24L01+ hardware setup, Enhanced ShockBurst protocol internals, the ping-pong time synchronization algorithm, dual-camera integration, error handling, complete peer implementation pseudocode, and performance analysis. It serves as a companion to the main Low-Cost-UMI technical report.

2 NRF24L01+ Hardware Overview

2.1 Module Variants

The NRF24L01+ is available in two common form factors:

- **Basic module** (green PCB, $\sim \$1$): On-board PCB antenna, 0 dBm TX power, ~ 10 m indoor range. Draws 11.3 mA at 0 dBm, powered directly from RPi 3.3V pin.
- **PA+LNA module** ($\sim \$3$): External antenna with power amplifier (PA) and low-noise amplifier (LNA), up to +20 dBm TX, ~ 100 m indoor range. Draws up to 115 mA at full power—this *exceeds the RPi 3.3V rail capacity* (50 mA max from GPIO header).

For bimanual grippers operating within ~ 3 m of each other, the basic module is sufficient.

2.2 Pin Mapping

Table 2: NRF24L01+ to Raspberry Pi 4B GPIO pin mapping. Uses SPI0 bus.

NRF Pin	Function	RPi GPIO	RPi Pin #	Notes
VCC	Power	3.3V	Pin 17	Add $10\mu\text{F} + 100\text{nF}$ bypass
GND	Ground	GND	Pin 20	
CE	Chip Enable	GPIO22	Pin 15	Active high: TX/RX enable
CSN	SPI Chip Select	GPIO8 (CE0)	Pin 24	Active low: SPI select
SCK	SPI Clock	GPIO11 (SCLK)	Pin 23	
MOSI	SPI Data In	GPIO10 (MOSI)	Pin 19	
MISO	SPI Data Out	GPIO9 (MISO)	Pin 21	
IRQ	Interrupt	GPIO25	Pin 22	Optional: active low

2.3 Wiring Schematic

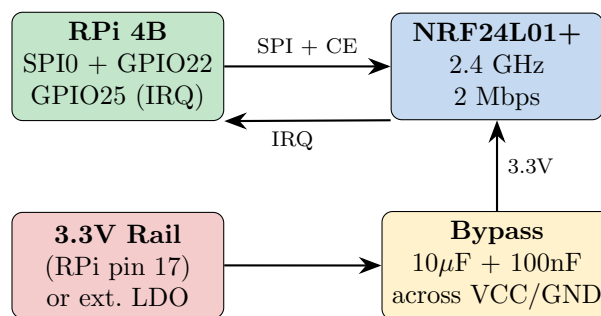


Figure 1: NRF24L01+ wiring to RPi 4B. Bypass capacitors are essential for stable operation.

PA+LNA Power Warning

The PA+LNA variant draws up to **115 mA at full TX power**. The RPi 3.3V GPIO pin can supply only ~ 50 mA. Powering a PA+LNA module directly from the RPi 3.3V rail will cause voltage drops, SPI communication failures, and unreliable radio operation. Use an **AMS1117-3.3V LDO** fed from the 5V rail (or LiPo UBEC output) with $10\mu\text{F}$ input and output capacitors.

2.4 SPI Configuration

Enable SPI on the Raspberry Pi:

```

1  # Method 1: raspi-config
2  sudo raspi-config nonint do_spi 0
3
4  # Method 2: /boot/config.txt
5  # Add or uncomment: dtparam=spi=on
6
7  # Verify SPI is active
8  ls /dev/spidev0.*
9  # Should show: /dev/spidev0.0 /dev/spidev0.1

```

Install the pyRF24 library:

```
1 pip install pyrf24
```

2.5 Updated Single-Gripper Hardware Architecture

Adding the NRF24L01+ module and a USB RGB camera to the existing gripper hardware:

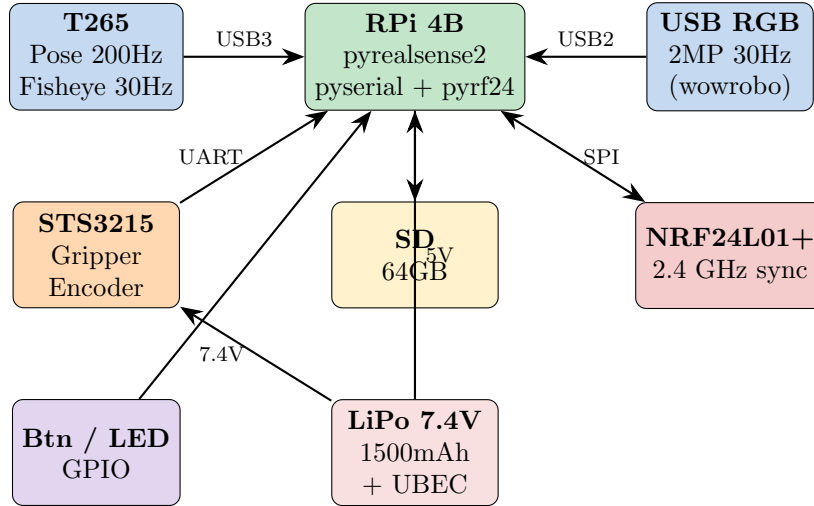


Figure 2: Updated single gripper hardware architecture with NRF24L01+ radio and USB RGB camera.

2.6 Updated Bill of Materials

Table 3: Updated BOM with NRF24L01+ and dual cameras. Adds ~\$36 total over WiFi-only system.

Component	Qty	Cost	Note
Intel RealSense T265	×2	~\$200 ea	eBay / AliExpress
Raspberry Pi 4B 4GB	×2	~\$55 ea	
MicroSD 64GB A2	×2	~\$12 ea	High write speed
Feetech STS3215	×2	~\$15 ea	12-bit encoder
USB RGB Camera (wowrobo)	×2	~\$15 ea	2MP, 30fps, 3m cable
NRF24L01+ module	×2	~\$1 ea	Basic PCB antenna
7.4V 2S LiPo 1500mAh	×2	~\$15 ea	XT30 + BMS
5V 3A UBEC	×2	~\$5 ea	
Misc + 3D housing	×2	~\$20 ea	PLA + TPU + caps
Bimanual total		~\$676	

3 Enhanced ShockBurst Protocol

The NRF24L01+ implements **Enhanced ShockBurst** (ESB), a hardware-accelerated packet protocol that handles preamble generation, address matching, CRC validation, automatic acknowledgement, and retransmission—all in silicon, without CPU intervention.

3.1 On-Air Packet Format

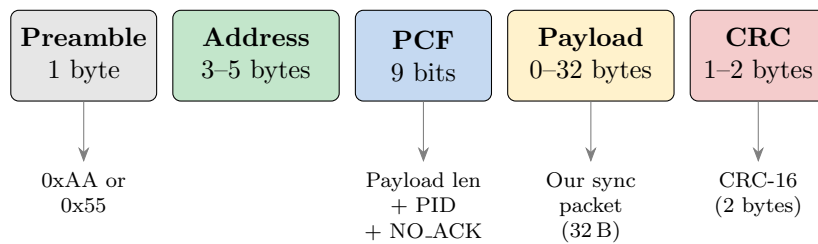


Figure 3: ESB on-air packet format. The Packet Control Field (PCF) is 9 bits packed before the payload.

3.2 Packet Control Field (PCF) Breakdown

Table 4: PCF field breakdown. PID increments per new payload to detect retransmitted duplicates.

Field	Bits	Range	Purpose
Payload Length	6	0–32	Dynamic payload size in bytes
PID	2	0–3	Packet ID for duplicate detection
NO_ACK	1	0/1	Per-packet ACK disable flag

3.3 Auto-ACK and Retransmission

When Enhanced ShockBurst is enabled (default), the transmitter automatically:

1. Sends the packet and switches to RX mode
2. Waits for an ACK from the receiver within the **Auto Retransmit Delay** (ARD)
3. If no ACK is received, retransmits up to **ARC** times (configurable 0–15)
4. Reports success or max-retries-exceeded to the application

The receiver, upon validating address and CRC, automatically sends an ACK packet. The PID field prevents the receiver from delivering duplicate payloads.

3.4 ESB Timing

Table 5: ESB timing for a single transmission at 2 Mbps with 32-byte payload.

	PLL Lock	TX Packet	RX Switch	Wait ACK	ACK RX
Duration	130 μ s	164.5 μ s	$\sim 10 \mu$ s	ARD	$\sim 30 \mu$ s
State	Standby	TX	TX→RX	RX	RX

3.5 Data Rate Selection

We select **2 Mbps** because our payloads are small (32 bytes) and range requirements are modest (~ 3 m). The shorter on-air time also reduces collision probability in the crowded 2.4 GHz band.

Table 6: Data rate comparison. We use 2 Mbps to minimize radio-on time and latency.

Data Rate	32B TX Time	On-Air Time	Use Case
250 kbps	~ 1.3 ms	~ 1.5 ms	Maximum range, lowest throughput
1 Mbps	~ 0.33 ms	~ 0.5 ms	Balanced
2 Mbps	~ 0.16 ms	~ 0.35 ms	Minimum latency (our choice)

3.6 ACK Payload Feature

The receiver can **pre-load a payload into the ACK packet**. When the transmitter sends a packet and the receiver auto-ACKs, the ACK carries the pre-loaded data back. This enables bidirectional data exchange in a single radio transaction.

The ACK payload must be loaded *before* the TX packet arrives. The ACK payload can be up to 32 bytes, same as a normal payload.

Channel Selection

The NRF24L01+ operates on channels 0–125 (2400–2525 MHz). WiFi channels 1, 6, and 11 occupy 2401–2473 MHz. To avoid interference, we use **channel 108** (2508 MHz), which is above the WiFi band. This is especially important if the RPi's WiFi is active for SSH or data transfer.

4 Ping-Pong Protocol Design

Time synchronization requires bidirectional communication: the initiator sends a timestamp, the responder replies with its own timestamps, and the initiator computes the clock offset.

4.1 Approach 1: Software Ping-Pong

In software ping-pong, each peer explicitly switches between TX and RX modes:

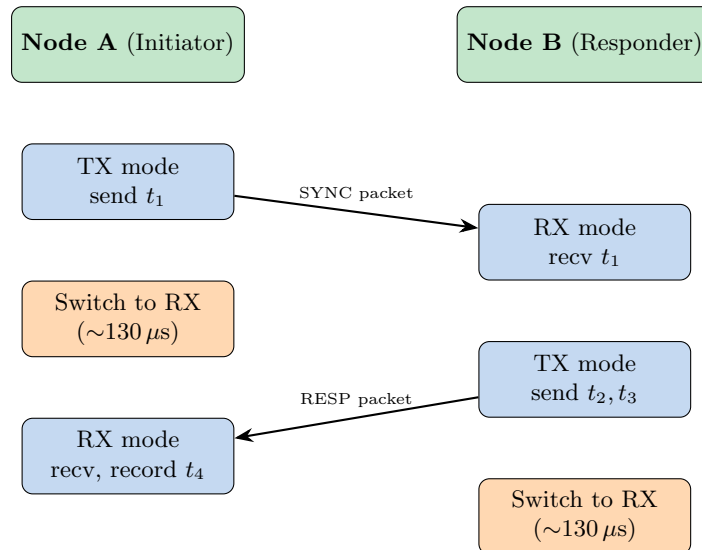


Figure 4: Software ping-pong: 4 mode switches per round trip. Each switch adds $\sim 130 \mu s$ PLL lock time.

4.2 Approach 2: Hardware ACK Payload

The ACK payload feature eliminates explicit mode switching. Node B pre-loads its response data into the ACK payload buffer:

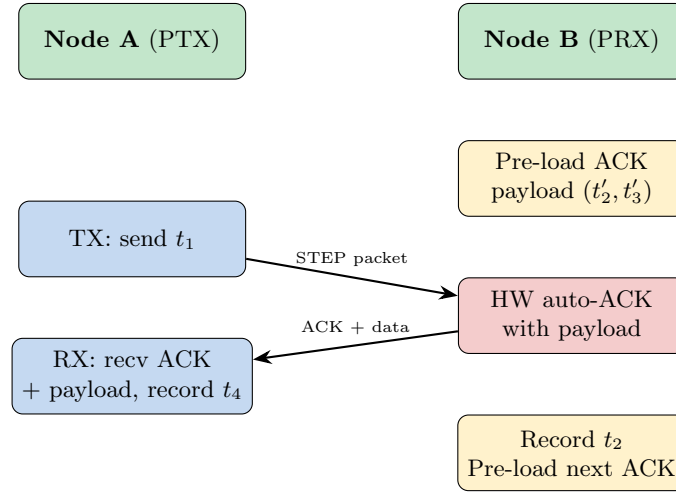


Figure 5: Hardware ACK payload: zero explicit mode switches. Node B stays in PRX mode permanently.

4.3 Approach Comparison

Table 7: We use hardware ACK payload for lower latency, accepting one-round-old timestamps.

Property	Software Ping-Pong	HW ACK Payload
Mode switches / round	4	0 (hardware)
Round-trip time	~ 1.5 ms	~ 0.6 ms
Implementation complexity	Medium	Low
Timestamp freshness	Current round	Previous round
Payload size each way	32 B	32 B
Requires	Mode switch logic	Pre-load before RX

We choose the **hardware ACK payload** approach. The one-round timestamp staleness (33 ms at 30 Hz) introduces negligible error: at 20 ppm clock drift, 33 ms contributes only 0.66 ns of additional offset—far below our measurement precision.

4.4 Pipe Configuration

The NRF24L01+ supports up to 6 receive pipes. We use pipe 1 for data and pipe 0 for auto-ACK reception:

```

1 from pyrf24 import RF24, RF24_PA_LOW, RF24_2MBPS
2
3 radio = RF24(22, 0)    # CE=GPI022, CSN=CE0
4 radio.begin()
5
6 # Shared addresses (5 bytes each)
7 ADDR_A = b"\xe7\xe7\xe7\xe7\xe7" # Node A TX, Node B RX pipe 1
8 ADDR_B = b"\xc2\xc2\xc2\xc2\xc2" # Node B TX, Node A RX pipe 1
9

```



```

10 # Node A (initiator / PTX):
11 radio.openWritingPipe(ADDR_A)      # TX on ADDR_A
12 radio.openReadingPipe(1, ADDR_B)   # RX on ADDR_B
13
14 # Node B (responder / PRX):
15 radio.openReadingPipe(1, ADDR_A)   # RX on ADDR_A
16 radio.openWritingPipe(ADDR_B)     # TX on ADDR_B

```

4.5 NRF24L01+ Packet Format

We extend the existing 28-byte UDP packet to a 32-byte NRF packet:

Table 8: 32-byte NRF packet format. Extended from 28-byte UDP with seq_num and reserved bytes.

Field	Type	Size	Offset	Role
magic	char[2]	2 B	0	“GS” identifier
msg_type	uint8	1 B	2	SYNC/STEP/STOP/ACK
step	uint32	4 B	3	Absolute step number
episode	uint16	2 B	7	Episode index
flags	uint8	1 B	9	REC / PAUSE / RESYNC
sender_id	uint8	1 B	10	Peer A=0, B=1
timestamp	float64	8 B	11	Sender monotonic clock (s)
sync_t2	float64	8 B	19	Responder rcv timestamp
seq_num	uint8	1 B	27	Sequence counter (0–255)
reserved	uint8[3]	3 B	28	Future use / padding
checksum	uint8	1 B	31	XOR of bytes 0–30
32 B				

Payload Size Trade-Off

The existing UDP protocol uses 28 bytes. Adding a sequence number (1 B) and keeping 3 B reserved for future extensions fills the 32-byte NRF maximum exactly. Since NRF24L01+ charges the same air time for any payload up to 32 bytes when using dynamic payloads, there is no penalty for using the full capacity.

5 Time Synchronization Algorithm

5.1 NTP-Style Four-Timestamp Principle

Clock synchronization between two peers requires estimating the **offset** θ between their clocks and the **one-way delay** δ .

5.2 Mathematical Derivation

Let $\theta = C_B - C_A$ be the true clock offset and assume symmetric one-way delay d :

$$t_2 = t_1 + d + \theta \qquad t_4 = t_3 + d - \theta \qquad (1)$$

Solving for θ :

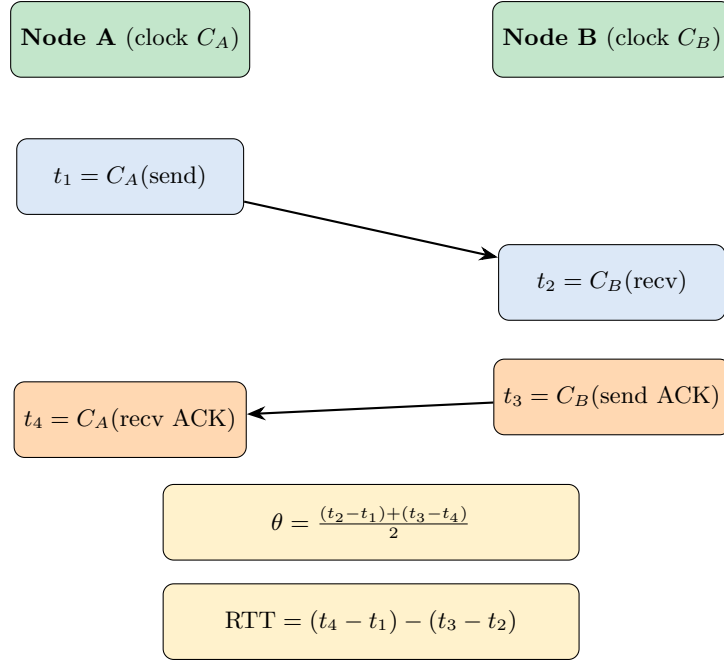


Figure 6: Four-timestamp exchange. t_1, t_4 measured on Node A's clock; t_2, t_3 on Node B's clock.

$$\theta = \frac{(t_2 - t_1) + (t_3 - t_4)}{2} \quad (2)$$

The round-trip time:

$$\text{RTT} = (t_4 - t_1) - (t_3 - t_2) \quad (3)$$

The one-way delay estimate:

$$d = \frac{\text{RTT}}{2} \quad (4)$$

5.3 Multi-Round Averaging with Outlier Rejection

We perform **10 rounds** and use median-based outlier rejection:

1. Collect 10 (θ_i, RTT_i) pairs
2. Compute $\tilde{\text{RTT}} = \text{median}(\text{RTT}_1, \dots, \text{RTT}_{10})$
3. Discard any round where $\text{RTT}_i > 1.5 \cdot \tilde{\text{RTT}}$
4. Final offset $\hat{\theta} = \text{mean}(\theta_i \text{ for kept rounds})$

5.4 Synchronization Algorithm

5.5 Worked Numerical Example

Use Monotonic Clock

Always use `time.monotonic()` (Python) or `CLOCK_MONOTONIC` (C). The wall clock `time.time()` can jump due to NTP adjustments. `time.monotonic()` has nanosecond resolution on Linux and is immune to wall-clock adjustments.

Algorithm 1: Time Synchronization — Initiator (Node A)

Input : radio (configured RF24), num_rounds = 10
Output: clock offset $\hat{\theta}$, mean RTT
offsets $\leftarrow []$, rtts $\leftarrow []$;
for $i \leftarrow 1$ **to** num_rounds **do**
 build SYNC packet with $t_1 \leftarrow \text{monotonic}()$;
 radio.write(packet) ; // TX + wait for ACK payload
 $t_4 \leftarrow \text{monotonic}()$;
 parse ACK payload: extract t_2, t_3 ;
 $\theta_i \leftarrow \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$;
 $RTT_i \leftarrow (t_4 - t_1) - (t_3 - t_2)$;
 append θ_i to offsets, RTT_i to rtts;
 sleep(5ms) ; // allow responder to pre-load next ACK
end
 $\tilde{RTT} \leftarrow \text{median}(\text{rtts})$;
foreach (θ_i, RTT_i) **do**
 if $RTT_i > 1.5 \cdot \tilde{RTT}$ **then**
 | discard θ_i
 end
end
 $\hat{\theta} \leftarrow \text{mean}(\text{kept offsets})$;
return $\hat{\theta}$, mean(kept RTTs);

Table 9: Worked example with 5 rounds. Median RTT = 0.80 ms, threshold = 1.20 ms. Round 3 rejected.

Round	$t_2 - t_1$ (ms)	$t_3 - t_4$ (ms)	θ_i (ms)	RTT (ms)	Keep?
1	0.42	-0.38	0.020	0.80	✓
2	0.41	-0.39	0.010	0.80	✓
3	0.85	-0.37	0.240	1.22	×
4	0.43	-0.40	0.015	0.83	✓
5	0.40	-0.38	0.010	0.78	✓
Result			0.014	0.80	

6 Dual-Camera Integration

6.1 Camera Overview

Each gripper unit now carries two cameras:

- **Intel RealSense T265:** 6DoF VIO at 200 Hz and fisheye stereo images at 30 Hz. Connected via USB 3.0.
- **USB RGB Camera (wowrobo):** 2MP resolution, 30 fps, connected via USB 2.0 with 3m cable.

6.2 Camera Specifications

Table 10: Dual camera specifications per gripper unit.

Camera	Resolution	FPS	FOV	Purpose
T265 (fisheye)	848×800	30	163°	Wide-angle observation + VIO pose
wowrobo RGB	1920×1080	30	~78°	Close-range color observation

The T265 uses USB 3.0 bandwidth. The wowrobo RGB camera uses USB 2.0 (sufficient for 1080p MJPEG@30fps). Since the RPi 4B has separate controllers, no bus contention exists.

6.3 Updated Software Architecture

Adding the USB RGB camera and NRF24L01+ radio creates a 5-thread architecture:

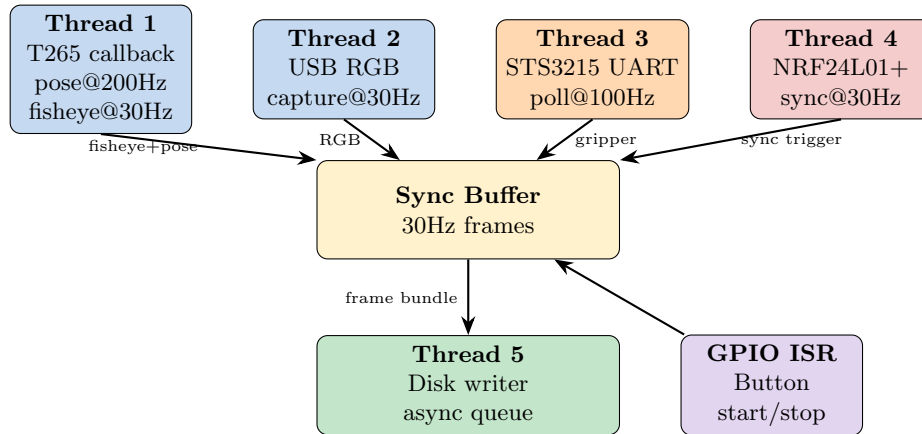


Figure 7: Updated 5-thread software architecture. Thread 4 (NRF) provides synchronization trigger; Thread 2 adds color observation.

6.4 Frame Synchronization Strategy

The NRF sync trigger from Thread 4 serves as the master clock:

1. **NRF trigger arrives** (Thread 4): Record t_{trigger} using `monotonic()`
2. **Latch T265 pose:** Nearest buffered pose (± 2.5 ms at 200 Hz)
3. **Latch T265 fisheye:** Nearest frame (± 16.7 ms at 30 Hz)

4. **Latch USB RGB frame:** Nearest frame (± 16.7 ms at 30 Hz)
5. **Latch STS3215:** Nearest reading (± 5 ms at 100 Hz)
6. **Bundle and enqueue** for disk writer

6.5 Updated LeRobot Dataset Columns

Table 11: Updated parquet columns with dual cameras and NRF sync diagnostics.

Column	Type	Shape	Description
observation.state	float32	(10,)	Pos (3) + quat (4) + gripper (1) + conf (1) + side (1)
observation.images.fisheye	video	(800, 848, 1)	T265 fisheye frame (MP4)
observation.images.rgb	video	(1080, 1920, 3)	USB RGB camera frame (MP4)
action	float32	(10,)	Target state for next step
timestamp	float64	—	Seconds from episode start
nrf_rtt_ms	float32	—	NRF round-trip time (ms)
clock_offset_ms	float32	—	Estimated clock offset (ms)
episode_index	int64	—	Episode identifier
frame_index	int64	—	Frame within episode
index	int64	—	Global frame index
task_index	int64	—	Links to tasks.jsonl

USB Bandwidth Budget

The RPi 4B has separate USB 3.0 (VL805) and USB 2.0 controllers. T265 uses USB 3.0 (~ 200 MB/s). The wowrobo RGB uses USB 2.0 (~ 40 MB/s; 1080p MJPEG@30fps needs ~ 15 MB/s). NRF24L01+ uses SPI (separate from USB). No bandwidth conflicts exist.

7 Error Handling and Drift Correction

7.1 Packet Loss Scenarios

Three failure tiers:

1. **Transient loss:** 1–2 packets lost, ESB auto-retry succeeds. Invisible to application. Adds ~ 0.5 ms per retry to RTT.
2. **Burst loss:** 3+ consecutive packets lost despite 15 retries. Software timeout fires (200 ms). Insert empty frame(s) and continue.
3. **Persistent failure:** 3 consecutive software timeouts. Enter RESYNC state, re-run time synchronization, resume recording.

7.2 Protocol State Machine

7.3 Retry and Timeout Configuration

7.4 Clock Drift Correction

Typical quartz crystals have ± 20 ppm tolerance:

$$\Delta\theta(t) = 20 \text{ ppm} \times t \quad (5)$$

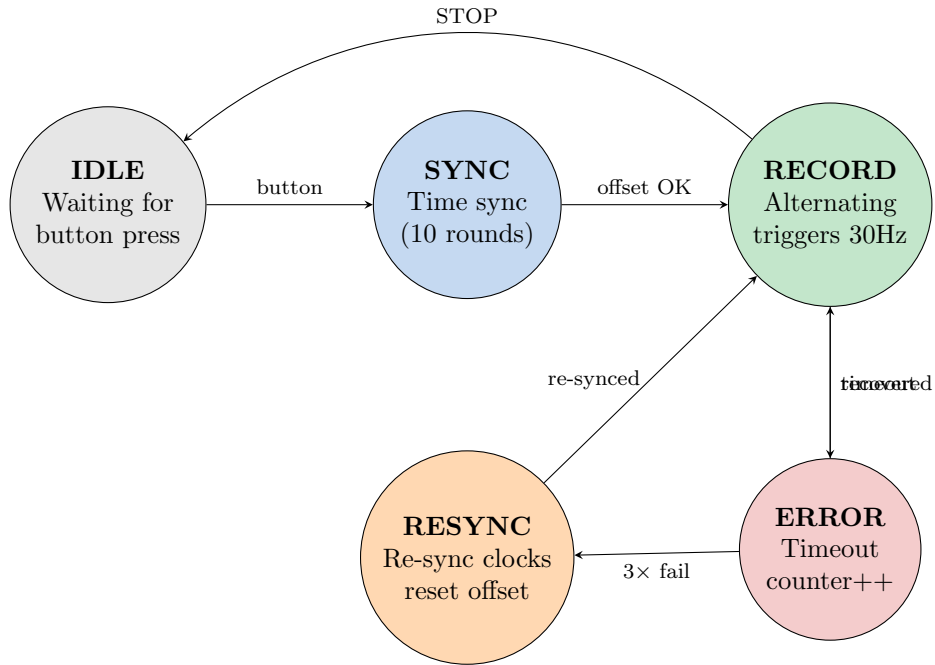


Figure 8: Protocol state machine. **ERROR** counts consecutive timeouts; 3 failures trigger **RESYNC**.

Table 12: Error handling configuration.

Parameter	Value	Rationale
Auto Retransmit Delay (ARD)	500 μ s	Must exceed ACK payload TX time (164 μ s at 2 Mbps)
Auto Retry Count (ARC)	15	Maximum hardware retries before reporting failure
Max HW retry time	$\sim$ 8 ms	$15 \times 500 \mu$ s + overhead
Software timeout	200 ms	$6 \times$ frame period; allows recovery within episode
RESYNC threshold	3 timeouts	3 consecutive SW timeouts $\rightarrow$ re-sync
Re-sync interval	300 steps	Periodic re-sync to correct drift (every 10 s at 30 Hz)

At 30 Hz, after 300 steps (10 seconds): $\Delta\theta = 20 \times 10^{-6} \times 10 = 0.2$ ms.

This is within our ± 0.4 ms sync error budget. By re-synchronizing every 300 steps, we keep accumulated drift below 0.2 ms. The re-sync is piggybacked on the normal STEP packet by setting the RESYNC flag.

For longer sessions, the maximum interval:

$$t_{\max} = \frac{\Delta\theta_{\max}}{20 \text{ ppm}} \quad (6)$$

For $\Delta\theta_{\max} = 0.1$ ms: $t_{\max} = 5$ s (150 steps).

7.5 Packet Loss Recovery

Algorithm 2: Packet Loss Recovery — Gap Detection

Input: received step number s_{new} , expected step s_{exp}

```

if  $s_{\text{new}} = s_{\text{exp}}$  then
    process frame normally;
     $s_{\text{exp}} \leftarrow s_{\text{exp}} + 1$ ;
else if  $s_{\text{new}} > s_{\text{exp}}$  then
     $\text{gap} \leftarrow s_{\text{new}} - s_{\text{exp}}$ ;
    for  $j \leftarrow 0$  to  $\text{gap} - 1$  do
        insert empty frame (copy previous state, mark as interpolated);
    end
    process  $s_{\text{new}}$  normally;
     $s_{\text{exp}} \leftarrow s_{\text{new}} + 1$ ;
    log warning: “gap of {gap} frames”;
else
    discard (duplicate or retransmit artifact);
end

```

SPI Failure Detection

If the NRF24L01+ STATUS register reads 0x00 or 0xFF, the SPI bus has failed (loose connection, power glitch, or module fault). Normal STATUS values are 0x0E (idle) or 0x2E (TX success). Check STATUS after every `radio.write()` and `radio.available()` call. On failure: log error, attempt `radio.begin()` re-init, and if that fails, fall back to unsynchronized recording.

8 Complete Peer Implementation

Both peers share the same radio initialization; only the main loop differs.

8.1 Radio Initialization (Shared)

8.2 Node A — Initiator Main Loop

8.3 Node B — Responder Main Loop

8.4 Packet Encoding

Algorithm 3: Radio Initialization — Both Peers

Input : `is_initiator` (bool), `CE_PIN = 22`, `CSN_PIN = 0`
Output: configured radio

```

radio ← RF24(CE_PIN, CSN_PIN);
radio.begin();
radio.setPALevel(RF24_PA_LOW); // 0 dBm
radio.setDataRate(RF24_2MBPS);
radio.setChannel(108); // above WiFi
radio.setRetries(1, 15); // ARD=500 μs, ARC=15
radio.setPayloadSize(32);
radio.enableAckPayload();
radio.enableDynamicPayloads();
ADDR_A ← b"\xe7...";
ADDR_B ← b"\xc2...";
if is_initiator then
    radio.openWritingPipe(ADDR_A);
    radio.openReadingPipe(1, ADDR_B);
else
    radio.openReadingPipe(1, ADDR_A);
    radio.openWritingPipe(ADDR_B);
end
radio.flush_tx(); radio.flush_rx();
return radio;

```

```

1  import struct
2
3  PACKET_FMT = "<2s B I H B B d d B 3s B"
4  assert struct.calcsize(PACKET_FMT) == 32
5
6  def pack_packet(msg_type, step, episode, flags, sender_id,
7                 timestamp, sync_t2, seq_num):
8      data = struct.pack(PACKET_FMT,
9                          b"GS", msg_type, step, episode, flags, sender_id,
10                          timestamp, sync_t2, seq_num, b"\x00\x00\x00", 0)
11      chk = 0
12      for b in data[:31]:
13          chk ^= b
14      return data[:31] + bytes([chk])
15
16  def unpack_packet(data):
17      fields = struct.unpack(PACKET_FMT, data)
18      chk = 0
19      for b in data[:31]:
20          chk ^= b
21      assert chk == fields[-1], "checksum mismatch"
22      return fields

```


Algorithm 4: Node A — Initiator (PTX) Main Loop

Input: radio, offset $\hat{\theta}$, fps = 30
step $\leftarrow 0$, timeout_count $\leftarrow 0$;
Pre-recording: run time sync (Alg. 1), get $\hat{\theta}$;
while *not STOP* **do**
 $t_{\text{start}} \leftarrow \text{monotonic}()$;
 capture sensors (T265, RGB, STS3215);
 build STEP packet: step, episode, $t_1 = \text{monotonic}()$;
 if $\text{step} \bmod 300 = 0$ **then**
 | set RESYNC flag
 end
 success $\leftarrow \text{radio.write}(\text{packet})$;
 $t_{\text{done}} \leftarrow \text{monotonic}()$;
 if *success and ACK payload available* **then**
 | parse ACK: remote state, t_2 , t_3 ;
 | **if** *RESYNC* **then**
 | update $\hat{\theta}$
 | **end**
 | store frame;
 | timeout_count $\leftarrow 0$;
 else
 | timeout_count++;
 | store with empty remote;
 | **if** $\text{timeout_count} \geq 3$ **then**
 | RESYNC
 | **end**
 end
 step++;
 sleep_until($t_{\text{start}} + 1/\text{fps}$);
end

Table 13: Latency breakdown for a single NRF round trip with 32-byte payload and ACK payload.

Phase	Duration	Cumulative	Notes
SPI TX (32B at 8 MHz)	$\sim 26 \mu\text{s}$	$26 \mu\text{s}$	$32 \text{ bytes} \times 8 \text{ bits} / 8 \text{ MHz}$
PLL lock	$130 \mu\text{s}$	$156 \mu\text{s}$	Crystal stabilization
On-air TX (2 Mbps)	$164.5 \mu\text{s}$	$320.5 \mu\text{s}$	Full packet / 2 Mbps
Receiver processing	$\sim 10 \mu\text{s}$	$330.5 \mu\text{s}$	Address match + CRC
ACK TX (2 Mbps, 32B)	$164.5 \mu\text{s}$	$495 \mu\text{s}$	ACK with payload
SPI RX (ACK payload)	$\sim 26 \mu\text{s}$	$521 \mu\text{s}$	Read from FIFO
Software overhead	$\sim 36 \mu\text{s}$	$557 \mu\text{s}$	Timestamp, pack/unpack
Theoretical RTT		$\sim 557 \mu\text{s}$	
Measured RTT		$\sim 800 \mu\text{s}$	Linux adds $\sim 243 \mu\text{s}$

Algorithm 5: Node B — Responder (PRX) Main Loop

```

Input: radio, offset  $\hat{\theta}$ 
radio.startListening();
Pre-load initial ACK payload;
while not STOP do
  if radio.available() then
     $t_2 \leftarrow \text{monotonic}()$ ;
    read packet; parse step, flags,  $t_1$ ;
    capture sensors;
    store frame;
    if RESYNC then
      | update  $\hat{\theta}$ 
    end
     $t_3 \leftarrow \text{monotonic}()$ ;
    radio.writeAckPayload(1, ack) ;           // for next round
  end
  else
    | sleep(100  $\mu\text{s}$ )
  end
end

```

9 Performance Analysis

9.1 Latency Breakdown

9.2 Synchronization Error Comparison

Table 14: Sync error comparison: WiFi UDP vs NRF24L01+.

Metric	WiFi UDP	NRF24L01+	Improvement
Mean RTT	~ 2.0 ms	~ 0.8 ms	$2.5\times$
RTT std dev	~ 0.5 ms	~ 0.15 ms	$3.3\times$
Sync error (10-round)	~ 0.5 ms	~ 0.2 ms	$2.5\times$
Max sync error	~ 1.5 ms	~ 0.4 ms	$3.7\times$
Drift after 300 steps	~ 0.2 ms	~ 0.2 ms	Same (crystal)
Error bound	Constant	Constant	Both bounded

9.3 Step Timing at 30 Hz

9.4 Power Consumption

Measured vs Theoretical

The $\sim 243 \mu\text{s}$ gap between theoretical ($557 \mu\text{s}$) and measured ($800 \mu\text{s}$) RTT is caused by Linux scheduling latency. The RPi 4B runs a non-realtime kernel; context switches add 50–200 μs jitter. This jitter is *symmetric*, so it cancels in the offset calculation. The offset estimate remains accurate to ± 0.2 ms after multi-round averaging.

Table 15: Single-step timing breakdown at 30 Hz (33.3 ms period). NRF duty cycle $\sim 2.4\%$.

	0–0.8 ms	0.8–1 ms	1–5 ms	5–32 ms	32–33.3 ms
NRF radio	TX+ACK	proc			
Sensors		latch	T265+RGB		
Disk				write	
Idle					sleep
Duty	$\sim 3\%$		$\sim 12\%$	$\sim 80\%$	$\sim 5\%$

Table 16: Power comparison. NRF24L01+ consumes $\sim 3000\times$ less power than WiFi.

State	NRF24L01+	WiFi (BCM43455)	Notes
TX active	11.3 mA	~ 300 mA	NRF at 0 dBm
RX active	13.5 mA	~ 100 mA	Listening
Standby-I	$26\ \mu\text{A}$	N/A	Between transactions
Power down	900 nA	N/A	Not used during recording
Average at 30 Hz	$\sim 50\ \mu\text{A}$	~ 150 mA	2.4% duty cycle
Daily energy	~ 4 mWh	~ 12 Wh	NRF: negligible

10 References and Hardware Links

10.1 QR Code Quick Links

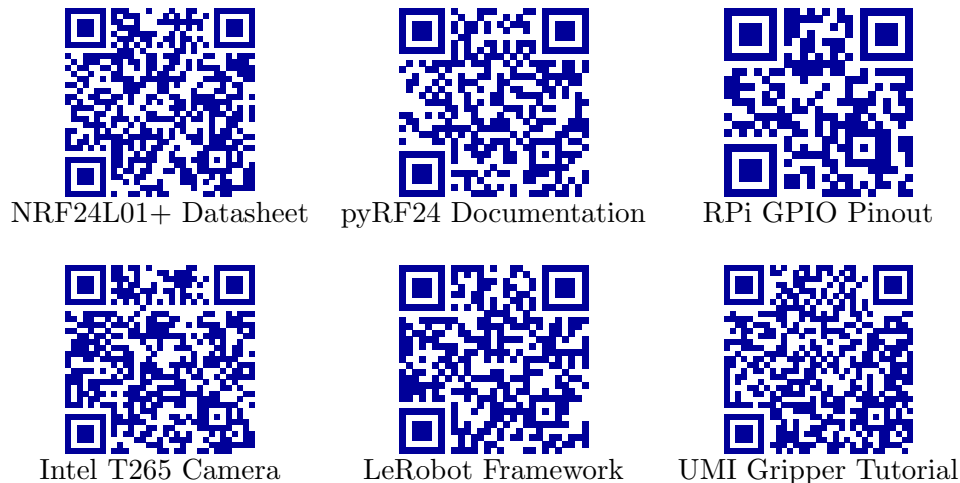


Figure 9: QR codes for key references.

10.2 Reference URLs

Resource	URL
NRF24L01+ Datasheet	nordicsemi.com/Products/nRF24-series
pyRF24 Library	nrf24.github.io/RF24/
RPi GPIO Pinout	pinout.xyz
Intel RealSense T265	intelrealsense.com/tracking-camera-t265/
wowrobo USB Camera	wowrobo.com (SO-ARM100/101 compatible)
LeRobot Framework	github.com/huggingface/lerobot
UMI Gripper (Stanford)	umi-gripper.github.io
Diffusion Policy	github.com/real-stanford/diffusion_policy
ACT (Aloha)	github.com/tonyzhaozh/act

Table 17: Complete reference URL list.